



TRAVAUX PRATIQUES N°1

API REST, HTTP et découverte du Model Context Protocol (MCP)

Étude de cas : intégration d'un système de ticketing TECHCORP

Élément	Indication
Public	4e année — étudiants Logiciel et Réseau
Organisation	Binômes ou trinômes mixtes. Au moins un profil Logiciel et un profil Réseau.
Durée	3 h 00
Outils	Python 3.11, FastAPI, Postman ou Swagger, MCP Inspector, Wireshark, Git/VS Code
Livrable	Un compte rendu commun par groupe, accompagné des captures demandées.

Mission confiée au groupe

Vous intervenez au sein de l'équipe « Architecture & Intégration » de TECHCORP. La direction souhaite préparer un futur assistant IA capable de consulter l'état du système d'information et les tickets d'incident. Avant toute connexion à un LLM, votre équipe doit prouver qu'elle maîtrise les échanges applicatifs et réseau qui permettront à cet assistant de fonctionner.

Règle du TP — Le travail est commun. Le profil Logiciel n'est pas uniquement chargé du code et le profil Réseau n'est pas uniquement chargé de Wireshark. Chaque étape comporte une vérification croisée : l'un exécute, l'autre explique, puis les rôles s'inversent.

1. Contexte de l'entreprise

TECHCORP fournit des services numériques à plusieurs clients internes. Son équipe d'exploitation maintient un petit parc de serveurs et une API de ticketing. Les incidents sont aujourd'hui consultés manuellement dans Swagger ou dans des exports JSON. La DSI veut, à terme, permettre à un assistant IA de répondre à des questions comme : « Quels tickets critiques sont encore ouverts ? », « Quel serveur est concerné ? » ou « Que savons-nous de SRV-DB-01 ? ».

Le projet n'en est qu'à sa première étape. Pour cette séance, aucun agent autonome n'est encore déployé. Le but est de comprendre trois éléments fondamentaux : comment une API REST expose des données, comment le réseau transporte ces échanges, et comment un premier serveur MCP expose une capacité à une application IA.

1.1 Architecture simplifiée du Jour 1

Le poste étudiant héberge localement l'API et le serveur MCP. Les serveurs TECHCORP ci-dessous sont, pour ce premier TP, des données métier simulées.

Composant	Rôle	Adresse / transport	Situation au début du TP
POSTE-ETU	Poste Windows du groupe	127.0.0.1 / loopback	Disponible
API-TICKETING	API FastAPI locale	127.0.0.1:8000 / HTTP	À démarrer
MCP-TECHCORP	Serveur MCP Python	STDIO via MCP Inspector	À démarrer
SRV-DB-01	Serveur base de données (simulé)	10.10.20.10:5432	UP dans le jeu de données
SRV-WEB-01	Serveur Web (simulé)	10.10.20.20:443	DEGRADED dans le jeu de données
SRV-DNS-01	Serveur DNS (simulé)	10.10.20.53:53	DOWN dans le jeu de données

Point d'attention — Un serveur marqué DOWN dans les données métier n'est pas automatiquement une machine réellement injoignable dans le laboratoire. Vous devrez distinguer ce que dit l'application de ce que vous avez effectivement observé sur le réseau.

1.2 Jeu de données métier fourni

ID	Titre	Serveur	Priorité	Statut
1	DNS indisponible	SRV-DNS-01	CRITICAL	OPEN
2	Latence application	SRV-WEB-01	HIGH	OPEN
3	Mise à jour PostgreSQL	SRV-DB-01	MEDIUM	CLOSED
4	Certificat à renouveler	SRV-WEB-01	HIGH	OPEN
5	Contrôle sauvegarde nocturne	SRV-DB-01	LOW	OPEN

2. Organisation du groupe

Le TP est conçu pour provoquer un dialogue technique entre les deux spécialités. Le binôme ne doit pas fonctionner comme deux étudiants réalisant deux travaux indépendants. À la fin de chaque phase, le membre qui n'était pas responsable de l'exécution doit expliquer ce qui vient d'être fait.

Séquence	Pilote principal	Contrôle croisé obligatoire
API REST	Profil Logiciel	Le profil Réseau explique l'IP, le port, le sens du flux et ce qu'il s'attend à voir dans Wireshark.
Capture Wireshark	Profil Réseau	Le profil Logiciel retrouve dans la capture l'endpoint, la méthode HTTP et le code de retour correspondant à son appel.
Serveur MCP	Profil Logiciel	Le profil Réseau explique pourquoi STDIO n'est pas équivalent à un flux TCP visible sur le réseau.
Synthèse	Commun	Les deux membres sont interrogés sur l'ensemble de la chaîne.

3. Objectifs pédagogiques

- Lire une architecture client / serveur et identifier les composants qui communiquent.
- Manipuler une API REST en GET et POST.
- Interpréter une requête HTTP, un code de réponse et un document JSON.
- Observer le même échange du point de vue réseau avec Wireshark.
- Découvrir MCP Inspector et exécuter un premier Tool MCP.
- Expliquer pourquoi MCP et REST sont complémentaires.
- Construire un schéma commun reliant la vue Logiciel et la vue Réseau.

4. Déroulement des 3 heures

Temps	Travail	Résultat attendu
0:00 – 0:20	Prise en main du contexte, lecture des données, schéma initial	Architecture annotée + répartition du groupe
0:20 – 1:05	Partie A — API REST et traitement des données	GET / POST testés + tableau d'analyse
1:05 – 1:50	Partie B — Analyse HTTP/TCP avec Wireshark	Capture commentée + réponses réseau
1:50 – 2:30	Partie C — Premier serveur MCP	Tool exécuté + protocole observé
2:30 – 3:00	Partie D — Mise en commun, schéma final et restitution orale	Livrable final du groupe

4.1 Travail préparatoire de 10 minutes

1. Entourez dans le tableau d'architecture ce qui est réel sur votre poste et ce qui est uniquement simulé.
2. À partir du jeu de tickets, déterminez sans outil : combien de tickets sont OPEN, combien sont HIGH ou CRITICAL, et quels serveurs sont concernés.
3. Sur une feuille, dessinez le flux que vous pensez observer lors de GET /tickets. Ce schéma sera comparé à Wireshark plus tard.

5. Partie A — Manipulation de l'API REST

DURÉE INDICATIVE : 45 MINUTES

Dans cette partie, vous travaillez d'abord du point de vue de l'application. Vous devez comprendre ce que l'API expose, comment un client l'interroge et quelle donnée est retournée.

5.1 Démarrer l'API

Depuis le dossier du projet fourni par l'enseignant, activez votre environnement Python puis démarrez FastAPI.

Commande de lancement

```
uvicorn api.app:app --reload --port 8000
```

Ouvrez ensuite Swagger dans un navigateur :

```
http://127.0.0.1:8000/docs
```

5.2 Vérifications avant les appels

4. Confirmez que Swagger est accessible.
5. Repérez les endpoints disponibles et leur méthode HTTP.
6. Identifiez les modèles JSON attendus par l'API.
7. Le profil Réseau note l'IP et le port utilisés par le serveur.

5.3 Appels obligatoires

N°	Requête	But	Preuve à conserver
A1	GET /tickets	Récupérer tous les tickets	Capture ou export de la réponse
A2	GET /status/open	Lister les tickets ouverts	Code HTTP + nombre de tickets
A3	POST /tickets	Créer un nouveau ticket	Body envoyé + réponse
A4	GET /tickets	Vérifier que le ticket créé apparaît	ID du ticket créé
A5	GET /endpoint-inexistant	Provoquer une erreur volontaire	Code HTTP observé

5.4 Ticket à créer

Corps JSON du POST

```
{
  "title": "Échec de sauvegarde nocturne",
  "server": "SRV-DB-01",
  "priority": "HIGH"
}
```

Consigne — Ne modifiez pas le contenu du ticket avant d'avoir réalisé la première capture. Le même ticket sera utilisé pour comparer la vision applicative et la vision réseau.

6. Partie A — Analyse des données et du contrat HTTP

Complétez ce tableau à partir de vos appels. Les réponses doivent être relevées dans Swagger, Postman ou curl, et non supposées.

Test	Méthode	URL / endpoint	Code HTTP	Type de contenu	Observation
A1					
A2					
A3					
A4					
A5					

6.1 Questions — dominante Logiciel

8. Dans le scénario TECHCORP, quelle différence fonctionnelle faites-vous entre GET et POST ?

9. Quels champs du ticket sont fournis par le client ? Quels champs pourraient être attribués par le serveur ?

10. Quel code de retour observez-vous lors de la création du ticket ? Que signifie-t-il ?

11. Dans GET /status/open, combien de tickets sont retournés après votre création ? Expliquez pourquoi ce nombre diffère ou non du jeu initial.

6.2 Questions — dominante Réseau

12. Pourquoi l'adresse 127.0.0.1 est-elle utilisée dans ce TP ?

13. Le client utilise-t-il lui aussi le port 8000 comme port source ? Formulez votre hypothèse avant la capture.

14. Si FastAPI écoutait uniquement sur 127.0.0.1, un autre PC du réseau pourrait-il y accéder directement ? Justifiez.

Point de contrôle croisé — Le profil Logiciel explique au profil Réseau où se trouve l'endpoint dans l'application. Le profil Réseau explique au profil Logiciel comment le même appel sera matérialisé par un flux IP/TCP/HTTP.

7. Partie B — Observer le flux avec Wireshark

DURÉE INDICATIVE : 45 MINUTES

Vous allez maintenant observer le même comportement avec une autre lecture : non plus le contrat API, mais les paquets réellement échangés sur le poste.

7.1 Préparer la capture

15. Lancez Wireshark.
16. Sélectionnez l'interface de loopback / Npcap Loopback Adapter.
17. Démarrez la capture.
18. Appliquez le filtre d'affichage ci-dessous.

Filtre Wireshark

```
tcp.port == 8000
```

19. Pendant la capture, relancez GET /status/open puis POST /tickets.

7.2 Éléments obligatoires à retrouver

- Le SYN émis par le client.
- Le SYN, ACK émis par le serveur.
- L'ACK final du three-way handshake.
- Le port source temporaire choisi par le client.
- Le port destination 8000.
- La requête HTTP GET ou POST.
- La réponse HTTP 200 OK ou 201 Created.
- Le contenu JSON lorsqu'il est visible en clair.

7.3 Tableau d'observation

Élément	Valeur observée	Interprétation du groupe
IP source		
IP destination		
Port source		
Port destination		
Méthode HTTP		
Endpoint		
Code HTTP		
Content-Type		

Bonus utile — Clic droit sur un paquet HTTP > Follow > TCP Stream. Comparez la conversation reconstruite par Wireshark avec ce que vous avez vu dans Swagger.

8. Partie B — Analyse croisée du flux

Cette page doit être traitée à deux voix. Le membre Réseau commence par expliquer la capture. Le membre Logiciel doit ensuite retrouver dans le code ou dans Swagger l'élément applicatif correspondant.

Observation	Question Réseau	Question Logiciel
Port 8000	Pourquoi ce port reste-t-il le même côté serveur ?	Où est-il défini dans la commande de lancement ?
Port source	Pourquoi change-t-il selon les connexions ?	Le code applicatif le choisit-il explicitement ?
GET /status/open	À quel moment apparaît la requête dans la capture ?	Quel endpoint traite cette route ?
200 / 201	Que prouve la réception du code de réponse ?	Quelle logique produit ce code ?
JSON	Pourquoi est-il lisible ici ?	Comment est-il utilisé côté application ?
FIN / ACK	Que signifie la fermeture de session ?	L'application a-t-elle encore quelque chose à envoyer ?

8.1 Questions d'analyse

20. Pourquoi l'IP source et l'IP destination sont-elles identiques dans cette capture ?

21. Quelle preuve avez-vous que l'API a réellement reçu votre requête ?

22. Une donnée métier qui dit « SRV-DNS-01 = DOWN » est-elle une preuve réseau ? Expliquez.

23. Si vous remplacez HTTP par HTTPS, qu'est-ce qui deviendrait moins lisible dans Wireshark ?

8.2 Schéma à produire dans le compte rendu

Dessinez le flux réel observé. Votre schéma doit contenir au minimum :

- POSTE-ETU / client HTTP
- 127.0.0.1
- port source temporaire
- port 8000
- GET ou POST
- réponse HTTP
- JSON

Ajoutez une légende distincte « visible dans Wireshark » et « visible surtout dans l'application ».

9. Partie C — Premier serveur MCP

DURÉE INDICATIVE : 40 MINUTES

Le but n'est pas encore de connecter MCP à PostgreSQL ou à une vraie machine distante. Vous allez d'abord comprendre le modèle d'exposition d'une capacité.

9.1 Fichier server.py fourni / à compléter

server.py — Tool de découverte

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("TechCorp")

SERVERS = {
    "SRV-DB-01": {"status": "UP", "service": "PostgreSQL"},
    "SRV-WEB-01": {"status": "DEGRADED", "service": "HTTPS"},
    "SRV-DNS-01": {"status": "DOWN", "service": "DNS"}
}

@mcp.tool()
def get_server_status(hostname: str) -> dict:
    hostname = hostname.upper()

    if hostname not in SERVERS:
        return {"success": False, "error": "Serveur inconnu"}

    return {
        "success": True,
        "hostname": hostname,
        **SERVERS[hostname]
    }
```

9.2 Lancer MCP Inspector

Commande de lancement

```
mcp dev server.py
```

Dans MCP Inspector :

- 24. Vérifiez que le serveur est Connected.
- 25. Ouvrez l’onglet Tools.
- 26. Exécutez get_server_status avec SRV-DB-01.
- 27. Recommencez avec SRV-DNS-01 puis avec SRV-UNKNOWN-01.
- 28. Ouvrez l’onglet Protocol et repérez tools/list puis tools/call.

9.3 Relevé

Appel	Résultat principal	Erreur / remarque
get_server_status("SRV-DB-01")		
get_server_status("SRV-DNS-01")		
get_server_status("SRV-UNKNOWN-01")		

10. Partie C — Comprendre ce que MCP fait réellement

10.1 Questions à traiter en binôme

29. Quel est le nom du Tool exposé ? Quel paramètre attend-il ?

30. Quelle différence voyez-vous entre tools/list et tools/call ?

31. Le Tool réalise-t-il un vrai ping ou une vraie connexion vers 10.10.20.10 ? Comment le savez-vous ?

32. Pourquoi cette différence entre « donnée simulée » et « mesure réelle » est-elle importante pour un futur assistant IA ?

33. MCP remplace-t-il l'API REST de la Partie A ? Justifiez avec l'architecture du TP.

10.2 Schéma MCP à reproduire et commenter

Schéma logique

```

MCP Inspector (client)
  |
  | tools/list / tools/call
  v
MCP Server : server.py
  |
  | exécute get_server_status()
  v
Données Python simulées
  |
  v
Résultat structuré vers MCP Inspector

```

Question Réseau — Ce flux MCP utilise ici STDIO entre le client et le serveur MCP. Il ne doit donc pas être confondu avec le flux TCP/HTTP observé dans Wireshark lors des appels FastAPI.

10.3 Extension rapide si le groupe avance

Ajoutez un Tool list_open_servers() qui retourne uniquement les serveurs dont l'état n'est pas DOWN. Le groupe doit :

- écrire la fonction ;
- la vérifier dans MCP Inspector ;
- expliquer si cela génère ou non un nouveau flux réseau réel.

11. Partie D — Mise en commun et synthèse

DURÉE INDICATIVE : 30 MINUTES

À ce stade, le groupe doit être capable de relier les deux visions. Les réponses ci-dessous sont communes : elles doivent être rédigées ensemble, pas réparties entre les spécialités.

11.1 Questions de synthèse

34. Quelle différence faites-vous entre une API REST et un serveur MCP dans ce TP ?

35. Quand vous appelez GET /status/open, quel flux réel apparaît dans Wireshark ?

36. Quand vous exécutez get_server_status() dans MCP Inspector avec STDIO, voyez-vous le même type de flux TCP ? Pourquoi ?

37. Pourquoi un futur LLM ne doit-il pas inventer l'état de SRV-DNS-01 ?

38. Comment feriez-vous, lors d'un prochain TP, pour transformer get_server_status() en Tool de diagnostic réel ?

39. Citez une information importante pour le développeur mais difficile à voir dans Wireshark, puis une information importante pour le Réseau mais absente du code métier.

11.2 Schéma final obligatoire

Le schéma remis doit contenir les deux chemins suivants :

- Chemin REST : Client → HTTP/TCP → FastAPI :8000 → JSON → Client.
- Chemin MCP : MCP Inspector → STDIO → server.py → Tool → donnée structurée → MCP Inspector.

Ajoutez, par une convention graphique de votre choix, les éléments réellement visibles dans Wireshark et ceux qui relèvent du niveau applicatif.

11.3 Restitution orale de 3 minutes

L'enseignant choisit au hasard un membre du groupe. Cette personne doit expliquer le schéma complet sans lire le compte rendu. Le second membre complète uniquement si nécessaire.

12. Livrable à remettre

Un seul document par groupe, au format PDF ou Word, de 3 à 5 pages hors annexes. Il doit contenir :

40. Une courte présentation de l'architecture testée.
41. Le tableau d'analyse des appels REST complété.
42. La capture du POST de création de ticket.
43. Une capture Wireshark annotée.
44. Une capture MCP Inspector montrant tools/call.
45. Le schéma final avec les vues REST et MCP.
46. Les réponses aux six questions de synthèse.

Qualité attendue — Les captures seules ne suffisent pas. Chaque image doit être accompagnée de 2 à 4 phrases

expliquant ce qu'elle prouve.

13. Barème indicatif

Critère	Éléments évalués	Points
API REST	Appels GET/POST, lecture des codes HTTP, compréhension du JSON	5
Analyse réseau	Capture propre, IP/ports, handshake, interprétation du flux	5
MCP	Serveur démarré, Tool exécuté, tools/list et tools/call compris	4
Analyse croisée	Capacité à relier les deux visions et à distinguer simulé / observé	3
Livrable et oral	Schéma, clarté, qualité des explications, travail réellement commun	3

14. Critères de réussite

- Vous savez expliquer le chemin d'une requête GET depuis le client jusqu'à FastAPI.
- Vous savez identifier ce même échange dans Wireshark.
- Vous comprenez que les données « UP / DOWN » de ce TP sont simulées.
- Vous savez expliquer ce qu'un Tool MCP expose à un client.
- Vous savez dire pourquoi STDIO n'est pas le même type de communication que TCP/HTTP.
- Les deux profils du groupe sont capables d'expliquer l'ensemble du TP.

Annexe A — Données de référence

Jeu de données minimal

```
SERVERS = {
  "SRV-DB-01": {"ip": "10.10.20.10", "role": "Database", "status": "UP"},
  "SRV-WEB-01": {"ip": "10.10.20.20", "role": "Web", "status": "DEGRADED"},
  "SRV-DNS-01": {"ip": "10.10.20.53", "role": "DNS", "status": "DOWN"}
}

TICKETS = [
  {"id": 1, "title": "DNS indisponible",
   "server": "SRV-DNS-01", "priority": "CRITICAL", "status": "OPEN"},

  {"id": 2, "title": "Latence application",
   "server": "SRV-WEB-01", "priority": "HIGH", "status": "OPEN"},

  {"id": 3, "title": "Mise à jour PostgreSQL",
   "server": "SRV-DB-01", "priority": "MEDIUM", "status": "CLOSED"},

  {"id": 4, "title": "Certificat à renouveler",
   "server": "SRV-WEB-01", "priority": "HIGH", "status": "OPEN"}
]
```

Annexe B — Commandes utiles

Objectif	Commande / filtre
Lancer FastAPI	uvicorn api.app:app --reload --port 8000
Swagger	http://127.0.0.1:8000/docs

Objectif	Commande / filtre
Filtre Wireshark	tcp.port == 8000
Lancer MCP Inspector	mcp dev server.py
Tester une URL en PowerShell	Invoke-WebRequest http://127.0.0.1:8000/tickets

Annexe C — Questions flash pour l'oral

- Pourquoi l'adresse source et l'adresse destination valent-elles toutes les deux 127.0.0.1 dans ce TP ?
- Pourquoi le client n'utilise-t-il pas lui aussi le port 8000 ?
- À quoi sert JSON dans l'API ?
- Pourquoi un Tool MCP n'est-il pas automatiquement une API REST ?
- Quelle différence entre « l'application dit DOWN » et « j'ai prouvé que le service est inaccessible » ?
- Pourquoi cette distinction est-elle importante avant de brancher un LLM au SI ?

Fin du TP1 — Conservez votre projet et vos captures : ils serviront de base aux TP suivants, où les Tools MCP seront progressivement connectés à de vrais services, à une base de données et à des serveurs Linux.